

SIMATS ENGINEERING



SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Python Programming

ASSIGNMENT CO_1

Name: M.Kushal

Course Code: CSA0824

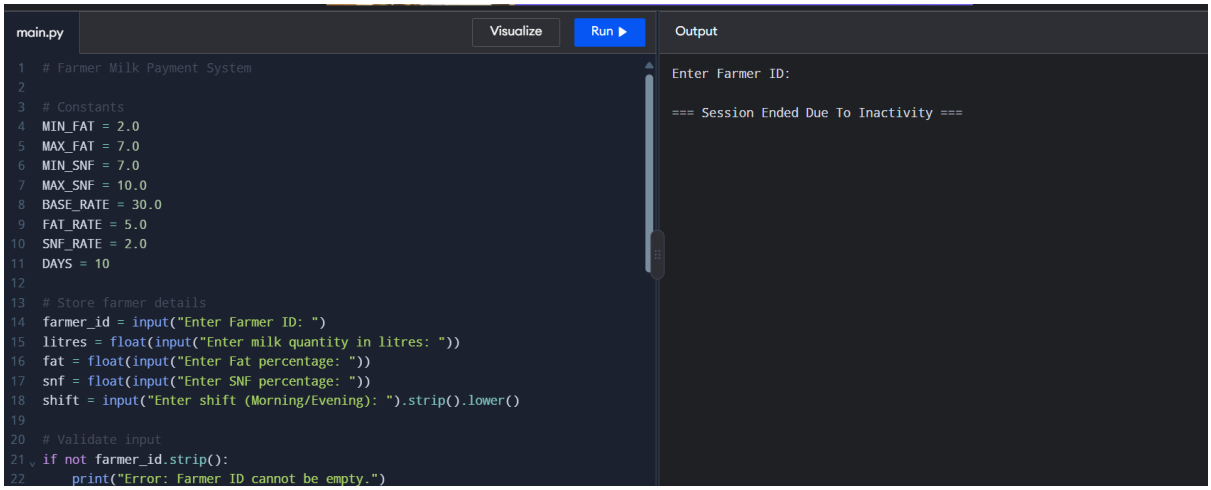
Subject: Python Programming

Slot: D

Department: B.Tech AI&DS

Github Link: https://github.com/mkushalkushal94-commits/CSA0824-Reg1924260243-Assignment1.gitkeep.git

TASK1



- Inputs: Farmer ID and shift are text; litres, fat percentage, and SNF percentage are numeric values.
- Outputs: Rate per litre, payment amount, and ten-day total are decimal values because payments can include paise.
- Constants: Minimum and maximum fat/SNF percentages, base rate, pricing coefficients, and ten-day period remain unchanged during a run.
- Rules: Litres cannot be negative, fat must be between 2% and 7%, SNF must be within the assumed range, and shift must be Morning or Evening.
- Assumption: The rate formula is illustrative, not an official dairy pricing formula. The ten-day total assumes the same quantity and milk quality every day.

Complexity analysis: For \(n\) farmer entries, processing takes \(O(n)\) time. Storing each entry in a list of records uses \(O(n)\) space. Using separate variables for every farmer would require an increasing number of manually named variables, making the program harder to maintain and scale to 300 farmers.

TASK2

main.py

```python
from decimal import Decimal
import sys

# TASK 2: Explore values and types

# 1. Check data types
print("----- DATA TYPES -----")
print(type(10))
print(type(10.0))
print(type("10"))
print(type(True))
print(type([10, 8.5]))

# 2. Explore division operators
print("\n----- DIVISION OPERATORS -----")
print("7 / 2 =", 7 / 2)
print("7 // 2 =", 7 // 2)
print("7 % 2 =", 7 % 2)
print("-7 // 2 =", -7 // 2)

# 3. Floating-point precision
print("\n----- FLOAT PRECISION -----")
print("0.1 + 0.2 =", 0.1 + 0.2)
print("0.1 + 0.2 == 0.3:", 0.1 + 0.2 == 0.3)

# 4. Create payment amounts
payments = ["33.40"] * 1000 + ["0.10"] * 3

# Using float
def total_float(items):
    total = 0.0
    for amount in items:
```

Console | I/O | AI Agent

```
----- FLOAT PRECISION -----
0.1 + 0.2 = 0.30000000000000004
0.1 + 0.2 == 0.3: False

----- PAYMENT TOTALS -----
Float: 33400.30000000061
Decimal: 33400.30

----- DECIMAL COMPARISON -----
False
True

----- DECIMAL CONVERSION -----
Decimal from string: 0.10
Decimal from float:
0.1000000000000000055511151231257827021181583404541015625

----- INTEGER PAISE -----
Total in paise: 3340030
Total in rupees: Rs. 33400.30

----- ROUNDED FLOAT -----
Rounded total: 33400.3

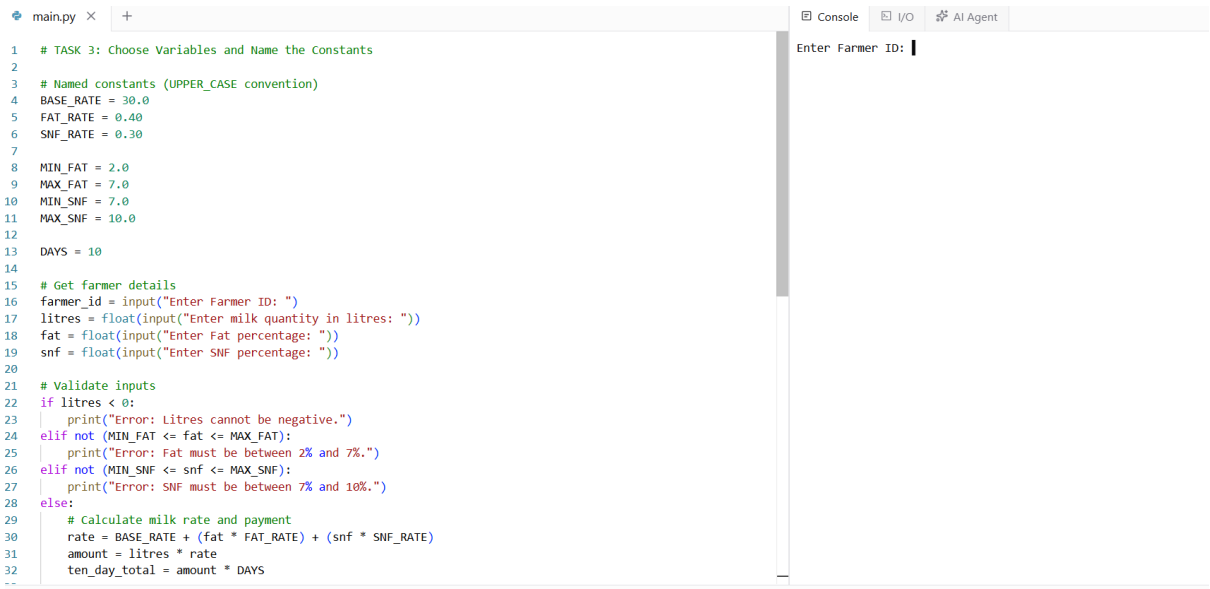
----- MEMORY USAGE -----
Integer: 28 bytes
Float: 24 bytes
String: 46 bytes
Decimal: 104 bytes
```

Important explanations for your record

1. / performs normal division, // performs floor division, and % returns the remainder.
2. -7 // 2 gives -4 because floor division rounds down toward negative infinity.
3. 0.1 + 0.2 == 0.3 is False due to floating-point representation.
4. Decimal("0.10") preserves the exact decimal value, making it suitable for money calculations.
5. Decimal(float("0.10")) preserves the float's existing approximation rather than the intended exact decimal.
6. Integer paise gives an exact total when all amounts are converted correctly into whole paise.
7. Processing \(n\) payments takes \(O(n)\) time. Storing the payment list takes \(O(n)\) space; the running total alone takes \(O(1)\) additional space.

Note: Your assignment's expected float output may differ slightly across Python versions, but the exact Decimal result should be 33400.30.

TASK3




```python
# TASK 3: Choose Variables and Name the Constants

# Named constants (UPPER_CASE convention)
BASE_RATE = 30.0
FAT_RATE = 0.40
SNF_RATE = 0.30

MIN_FAT = 2.0
MAX_FAT = 7.0
MIN_SNF = 7.0
MAX_SNF = 10.0

DAYS = 10

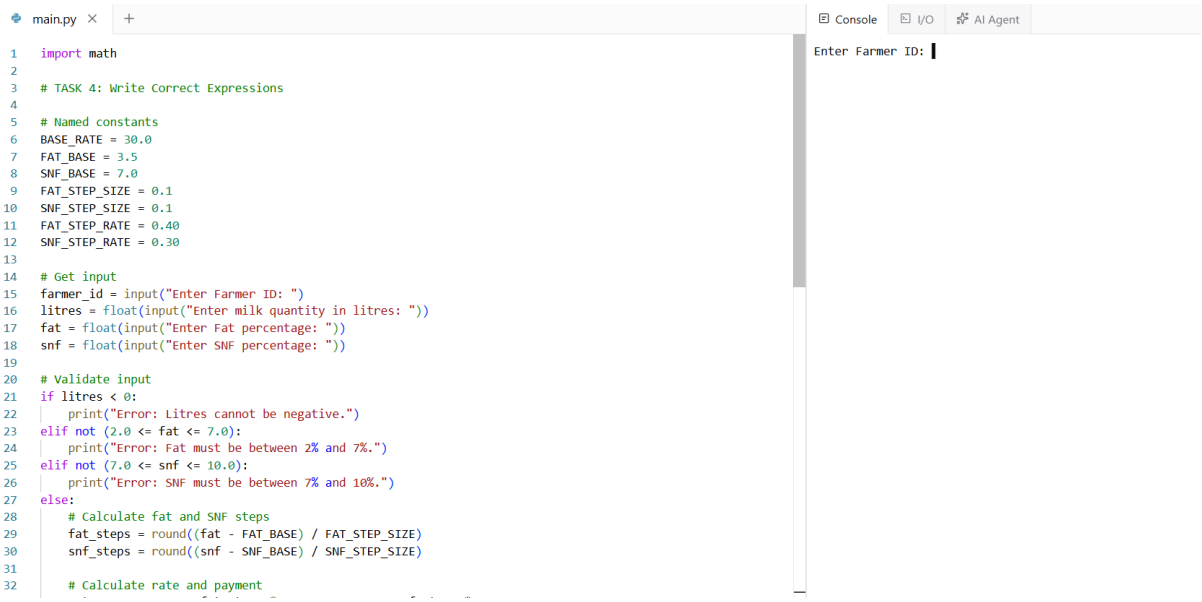
# Get farmer details
farmer_id = input("Enter Farmer ID: ")
litres = float(input("Enter milk quantity in litres: "))
fat = float(input("Enter Fat percentage: "))
snf = float(input("Enter SNF percentage: "))

# Validate inputs
if litres < 0:
    print("Error: Litres cannot be negative.")
elif not (MIN_FAT <= fat <= MAX_FAT):
    print("Error: Fat must be between 2% and 7%.")
elif not (MIN_SNF <= snf <= MAX_SNF):
    print("Error: SNF must be between 7% and 10%.")
else:
    # Calculate milk rate and payment
    rate = BASE_RATE + (fat * FAT_RATE) + (snf * SNF_RATE)
    amount = litres * rate
    ten_day_total = amount * DAYS
```

1. Naming conventions: Variables use lowercase names with underscores, such as farmer_id. Constants use uppercase names, such as BASE_RATE.
2. Constants: Python does not enforce true constants. Uppercase naming is a convention that tells programmers not to change those values.
3. Dynamic typing: A variable can store an integer and later store a string because Python determines its type at runtime.
4. Multiple assignment: a, b = b, a swaps two values without a temporary variable.
5. Alternative design: Named constants allow a rate revision in one place. A dictionary such as RATE_CARD groups related rates together. Hard-coded values repeated in \(k\) places may require \(k\) edits, while named constants require one edit. Neither approach meaningfully changes the program's \(O(n)\) processing time for \(n\) farmers.

Note: The rates in this program are illustrative examples based on the task's rate card, not official dairy payment rates.

TASK4




```python
import math

# TASK 4: Write Correct Expressions

# Named constants
BASE_RATE = 30.0
FAT_BASE = 3.5
SNF_BASE = 7.0
FAT_STEP_SIZE = 0.1
SNF_STEP_SIZE = 0.1
FAT_STEP_RATE = 0.40
SNF_STEP_RATE = 0.30

# Get input
farmer_id = input("Enter Farmer ID: ")
litres = float(input("Enter milk quantity in litres: "))
fat = float(input("Enter Fat percentage: "))
snf = float(input("Enter SNF percentage: "))

# Validate input
if litres < 0:
    print("Error: Litres cannot be negative.")
elif not (2.0 <= fat <= 7.0):
    print("Error: Fat must be between 2% and 7%.")
elif not (7.0 <= snf <= 10.0):
    print("Error: SNF must be between 7% and 10%.")
else:
    # Calculate fat and SNF steps
    fat_steps = round((fat - FAT_BASE) / FAT_STEP_SIZE)
    snf_steps = round((snf - SNF_BASE) / SNF_STEP_SIZE)

    # Calculate rate and payment
```

1. Operator precedence: Exponentiation is evaluated from right to left. Unary minus is applied after exponentiation in -2 ** 2.
2. Floating-point error: Decimal fractions may not be represented exactly in binary. Using round() gives the intended whole-number step count in this example.
3. Rounding differences: round() rounds to the nearest integer, int() truncates toward zero, and math.floor() rounds down toward negative infinity.
4. Comparison chaining: 2.0 <= fat <= 7.0 checks both limits. Boolean operators such as and, or, and not combine conditions.
5. Unexpected results: 0.1 + 0.2 is not exactly 0.3; (4.2 - 3.5) / 0.1 is slightly above 7; and round(2.675, 2) produces 2.67 because of binary floating-point representation.

Complexity: Each calculation uses \(O(1)\) time and \(O(1)\) extra space per farmer. The rates and step sizes are illustrative and should be adjusted to your dairy's official pricing rules.

TASK5




```python
# TASK 5: Read Input and Convert Types

def read_number(prompt):
    while True:
        text = input(prompt).strip()

        # Validate and convert input
        try:
            value = float(text)
            if value < 0:
                print("Error: Enter a non-negative number.")
                continue
            return value
        except ValueError:
            print("Invalid input! Enter a number such as 8.5.")

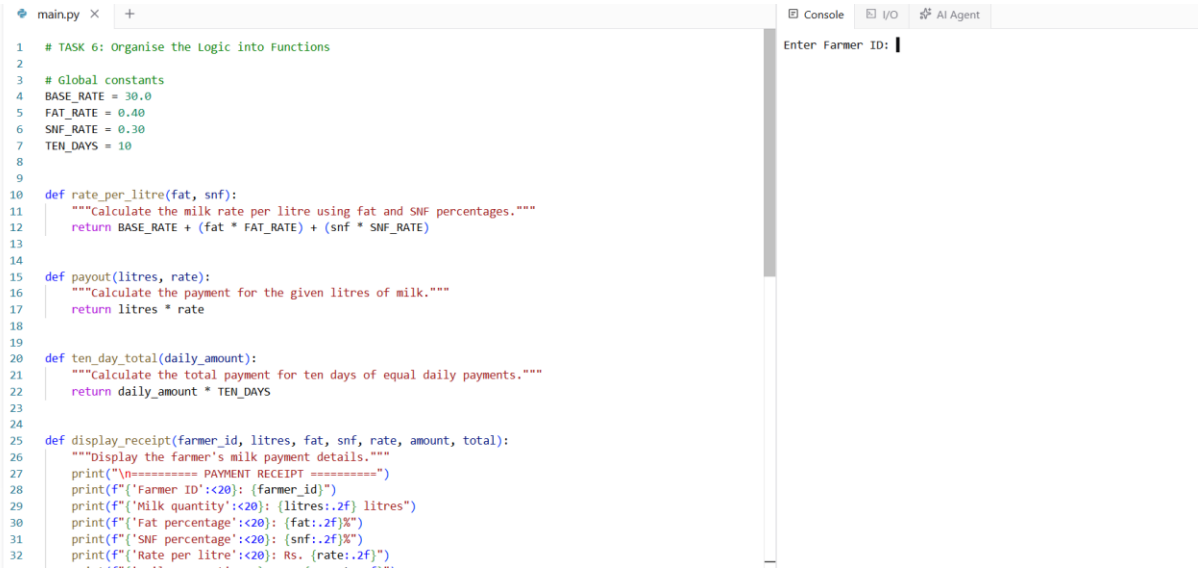

# Read farmer details
farmer_id = input("Enter Farmer ID: ")
litres = read_number("Enter milk quantity in litres: ")
fat = read_number("Enter Fat percentage: ")
snf = read_number("Enter SNF percentage: ")

# Validate ranges
if not (2.0 <= fat <= 7.0):
    print("Error: Fat must be between 2% and 7%.")
elif not (7.0 <= snf <= 10.0):
    print("Error: SNF must be between 7% and 10%.")
else:
    # Example pricing formula
    BASE_RATE = 30.0
    FAT_RATE = 0.40
```

1. Input conversion: input() always returns a string. float() converts numeric text into a decimal number.
2. Invalid input: Values such as "eight" or "8,5" cause ValueError when passed to float(). The try-except block catches the error and asks again. Enter 8.5 instead.
3. Formatted output: F-strings such as {amount:8.2f} display numbers with two decimal places and a field width of eight characters.
4. Alternative approaches: String checks can reject some invalid entries before conversion, but try-except ValueError is more reliable for general numeric input. Both approaches take approximately \(O(1)\) time per ordinary input, excluding repeated retries.

Note: The milk pricing formula is illustrative. Replace the rates with your assignment's official values if provided.

TASK6




```python
# TASK 6: Organise the Logic into Functions

# Global constants
BASE_RATE = 30.0
FAT_RATE = 0.40
SNF_RATE = 0.30
TEN_DAYS = 10


def rate_per_litre(fat, snf):
    """Calculate the milk rate per litre using fat and SNF percentages."""
    return BASE_RATE + (fat * FAT_RATE) + (snf * SNF_RATE)


def payout(litres, rate):
    """Calculate the payment for the given litres of milk."""
    return litres * rate


def ten_day_total(daily_amount):
    """Calculate the total payment for ten days of equal daily payments."""
    return daily_amount * TEN_DAYS


def display_receipt(farmer_id, litres, fat, snf, rate, amount, total):
    """Display the farmer's milk payment details."""
    print("\n========== PAYMENT RECEIPT ==========")
    print(f"{'Farmer ID':<20}: {farmer_id}")
    print(f"{'Milk quantity':<20}: {litres:.2f} litres")
    print(f"{'Fat percentage':<20}: {fat:.2f}%")
    print(f"{'SNF percentage':<20}: {snf:.2f}%")
    print(f"{'Rate per litre':<20}: Rs. {rate:.2f}")
```

1. Functions: rate_per_litre(), payout(), and ten_day_total() divide the calculation into reusable parts.
2. Return vs print: return sends a result back to the calling code so it can be reused in further calculations. print() only displays information.
3. Local and global variables: Variables such as litres and rate are local to their functions. Constants such as BASE_RATE and FAT_RATE are global and can be read inside functions without changing them.
4. Docstrings: Each function has a short description explaining its purpose.
5. Alternative design and complexity: One long script may repeat calculations, while functions improve readability and reuse. Lambda expressions are suitable for small expressions but are less readable for complex logic. For \(n\) farmer entries, processing takes \(O(n)\) time; function-call overhead is generally small, and each individual calculation uses \(O(1)\) extra space.

Note: The rates used are illustrative. Use your assignment's official pricing formula if one is provided.

TASK7




```python
# TASK 5: Read Input and Convert Types

def read_number(prompt):
    while True:
        text = input(prompt).strip()

        # Validate and convert input
        try:
            value = float(text)
            if value < 0:
                print("Error: Enter a non-negative number.")
                continue
            return value
        except ValueError:
            print("Invalid input! Enter a number such as 8.5.")

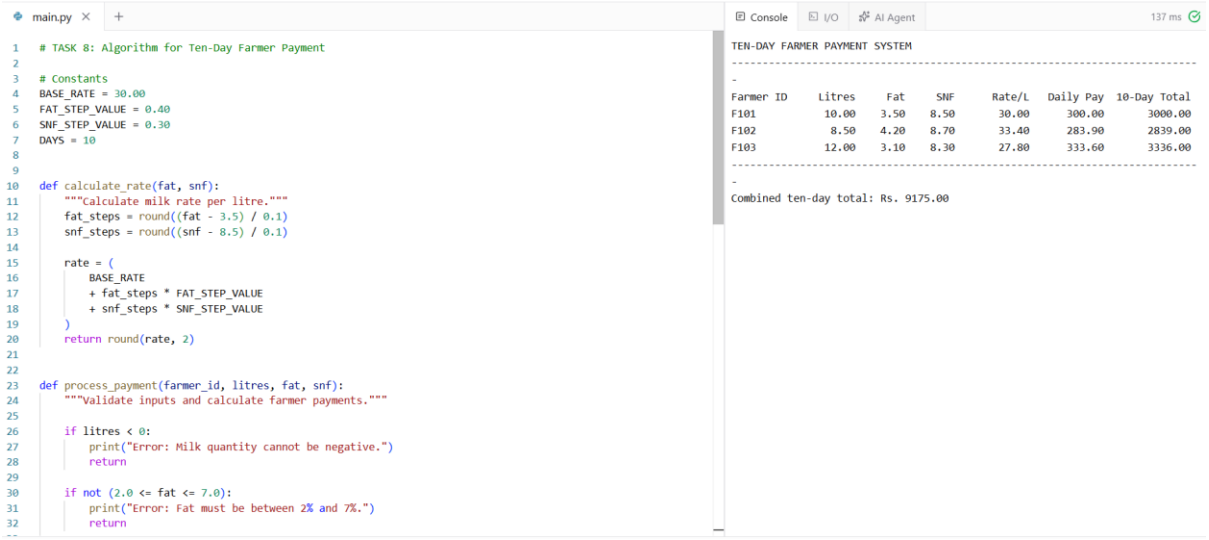

# Read farmer details
farmer_id = input("Enter Farmer ID: ")
litres = read_number("Enter milk quantity in litres: ")
fat = read_number("Enter Fat percentage: ")
snf = read_number("Enter SNF percentage: ")

# Validate ranges
if not (2.0 <= fat <= 7.0):
    print("Error: Fat must be between 2% and 7%.")
elif not (7.0 <= snf <= 10.0):
    print("Error: SNF must be between 7% and 10%.")
else:
    # Example pricing formula
    BASE_RATE = 30.0
    FAT_RATE = 0.40
```

1. Import methods: import milkrate uses milkrate.payout(), from milkrate import payout uses payout() directly, and import milkrate as mr uses mr.payout().
2. if __name__ == "__main__": runs the test table only when milkrate.py is executed directly. When another script imports it, the table is not printed.
3. Standard modules: math provides mathematical functions, and decimal supports precise decimal arithmetic for money calculations.
4. Module search: Python searches locations listed in sys.path, typically including the script's directory and installed library directories.
5. Complexity: For \(n\) farmer entries, processing is \(O(n)\). A module has a small one-time import cost and is cached in sys.modules; reusing it avoids copying and maintaining the same calculation code in multiple scripts.

Important: Keep both files in the same folder and run milkrate.py first to see the table. Then run test_milkrate.py to verify that importing the module works.

TASK8



```python
# TASK 8: Algorithm for Ten-Day Farmer Payment

# Constants
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
DAYS = 10


def calculate_rate(fat, snf):
    """Calculate milk rate per litre."""
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)

    rate = (
        BASE_RATE
        + fat_steps * FAT_STEP_VALUE
        + snf_steps * SNF_STEP_VALUE
    )
    return round(rate, 2)


def process_payment(farmer_id, litres, fat, snf):
    """Validate inputs and calculate farmer payments."""

    if litres < 0:
        print("Error: Milk quantity cannot be negative.")
        return

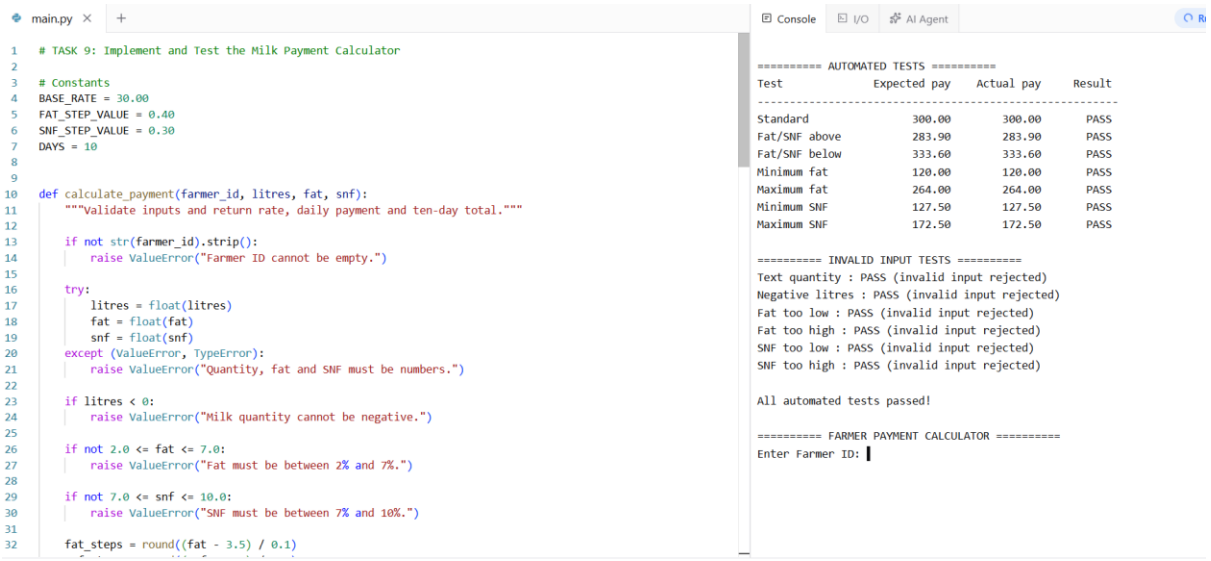
    if not (2.0 <= fat <= 7.0):
        print("Error: Fat must be between 2% and 7%.")
        return
```

```
TEN-DAY FARMER PAYMENT SYSTEM
------------------------------------------------------------------------------
-
Farmer ID     Litres     Fat     SNF      Rate/L   Daily Pay  10-Day Total
F101           10.00    3.50    8.50       30.00      300.00       3000.00
F102            8.50    4.20    8.70       33.40      283.90       2839.00
F103           12.00    3.10    8.30       27.80      333.60       3336.00
------------------------------------------------------------------------------
-
Combined ten-day total: Rs. 9175.00
```

Complexity: Processing \(n\) farmers takes \(O(n)\) time. The list of farmer records uses \(O(n)\) space; a running-total approach can reduce additional space to \(O(1)\).

Note: The ten-day total assumes each farmer supplies the same quantity and milk quality every day.

TASK9



```
main.py

# TASK 9: Implement and Test the Milk Payment Calculator

# Constants
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
DAYS = 10


def calculate_payment(farmer_id, litres, fat, snf):
    """Validate inputs and return rate, daily payment and ten-day total."""

    if not str(farmer_id).strip():
        raise ValueError("Farmer ID cannot be empty.")

    try:
        litres = float(litres)
        fat = float(fat)
        snf = float(snf)
    except (ValueError, TypeError):
        raise ValueError("Quantity, fat and SNF must be numbers.")

    if litres < 0:
        raise ValueError("Milk quantity cannot be negative.")

    if not 2.0 <= fat <= 7.0:
        raise ValueError("Fat must be between 2% and 7%.")

    if not 7.0 <= snf <= 10.0:
        raise ValueError("SNF must be between 7% and 10%.")

    fat_steps = round((fat - 3.5) / 0.1)
```

```
Console   I/O   AI Agent

========== AUTOMATED TESTS ==========
Test              Expected pay    Actual pay      Result
------------------------------------------------------------
Standard                300.00        300.00       PASS
Fat/SNF above           283.90        283.90       PASS
Fat/SNF below           333.60        333.60       PASS
Minimum fat             120.00        120.00       PASS
Maximum fat             264.00        264.00       PASS
Minimum SNF             127.50        127.50       PASS
Maximum SNF             172.50        172.50       PASS

========== INVALID INPUT TESTS ==========
Text quantity : PASS (invalid input rejected)
Negative litres : PASS (invalid input rejected)
Fat too low : PASS (invalid input rejected)
Fat too high : PASS (invalid input rejected)
SNF too low : PASS (invalid input rejected)
SNF too high : PASS (invalid input rejected)

All automated tests passed!

========== FARMER PAYMENT CALCULATOR ==========
Enter Farmer ID:
```

The program then asks for a farmer's details and displays the payment receipt.

Five-entry hand calculation: Using the first five valid entries in the test list, the combined daily payment is ₹1,301.50, and the combined ten-day total is ₹13,015.00, assuming the same daily quantity and quality.

Complexity analysis: Each payment calculation takes O(1) time and O(1)extra space. Automated testing takes O(n)time for n test cases. Using assert statements makes it easier to rerun all tests after changing the pricing formula.

TASK10



- Decimal calculates the exact decimal total when values are created from strings. float can accumulate small binary representation errors.
- Execution times vary with the computer, Python version, and system load. Use the measured times printed by your program for your record.
- Rounding each payment can produce a different result from rounding only the final total. In this example, each ₹1.005 payment is rounded to ₹1.01, while the exact sum is rounded only once.
- A dictionary rate table avoids recalculating rates for repeated fat/SNF pairs. The benefit is greatest when many entries share the same pairs.

Complexity analysis: Totalling O(n)payments takes O(1) time. A running total uses O(n) extra space, while storing every payment uses O(K) space. A rate table uses O(K) space for K unique pairs and provides average O(1) lookup per entry.

TASK11

```python
# TASK 11: Recommend and Defend the Final Design

import json
import os
import time
from decimal import Decimal, ROUND_HALF_UP

# Default rate card stored outside the program
CONFIG_FILE = "rate_card.json"

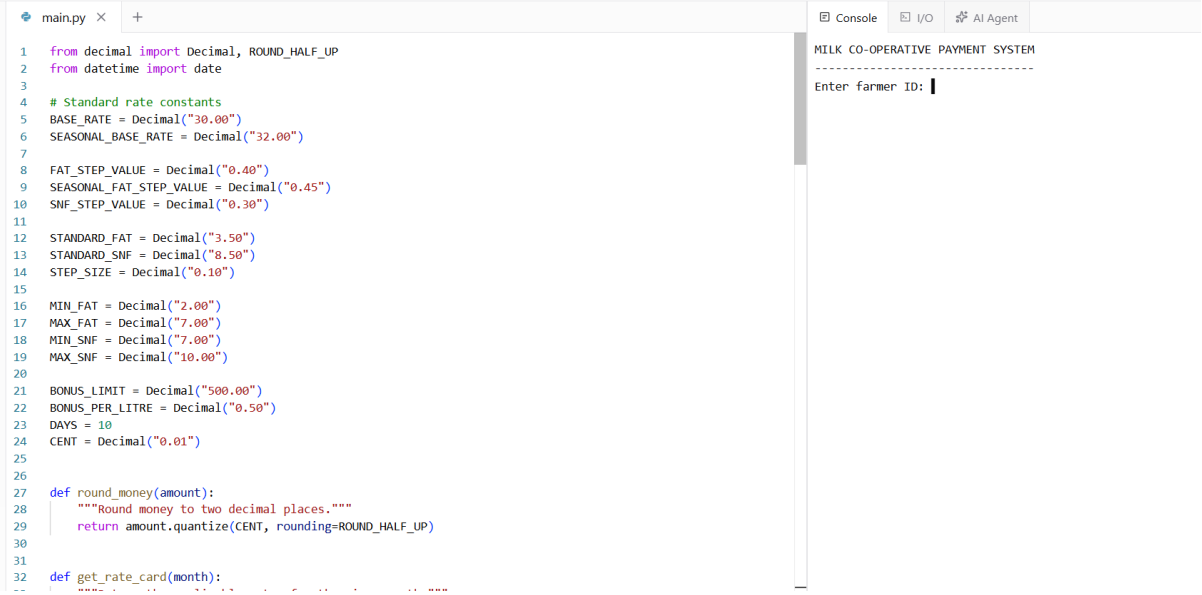
DEFAULT_RATE_CARD = {
    "base_rate": "30.00",
    "fat_step_value": "0.40",
    "snf_step_value": "0.30",
    "fat_standard": "3.5",
    "snf_standard": "8.5",
    "step_size": "0.1",
    "min_fat": "2.0",
    "max_fat": "7.0",
    "min_snf": "7.0",
    "max_snf": "10.0",
    "days": 10
}
```

```
VILLAGE MILK CO-OPERATIVE PAYMENT SYSTEM
Rate card load time: 0.000036 seconds
Enter Farmer ID:
```

1. Data types and rounding: Use Decimal for money and round each payable amount to two decimal places using ROUND_HALF_UP.
2. Rate revision: Edit rate_card.json instead of changing the Python code. Restart the program to load the updated rates.
3. Limitations: Invalid configuration values must be checked, and the program currently handles one farmer at a time. Production systems should also store transaction records and maintain backups.
4. District-union scale: The design can process many entries, but a large deployment needs a database, access controls, audit logs, and transaction handling.
5. Complexity analysis: Processing \(n\) farmers takes \(O(n)\) time. The rate card occupies \(O(k)\) space for \(k\) settings or rate entries. Loading the JSON file adds a small startup cost but avoids editing and redistributing the program after every rate revision.

Important: Run this program in a Python environment that allows file creation. It automatically creates rate_card.json on its first run.

TASK12




```python
from decimal import Decimal, ROUND_HALF_UP
from datetime import date

# Standard rate constants
BASE_RATE = Decimal("30.00")
SEASONAL_BASE_RATE = Decimal("32.00")

FAT_STEP_VALUE = Decimal("0.40")
SEASONAL_FAT_STEP_VALUE = Decimal("0.45")
SNF_STEP_VALUE = Decimal("0.30")

STANDARD_FAT = Decimal("3.50")
STANDARD_SNF = Decimal("8.50")
STEP_SIZE = Decimal("0.10")

MIN_FAT = Decimal("2.00")
MAX_FAT = Decimal("7.00")
MIN_SNF = Decimal("7.00")
MAX_SNF = Decimal("10.00")

BONUS_LIMIT = Decimal("500.00")
BONUS_PER_LITRE = Decimal("0.50")
DAYS = 10
CENT = Decimal("0.01")


def round_money(amount):
    """Round money to two decimal places."""
    return amount.quantize(CENT, rounding=ROUND_HALF_UP)


def get_rate_card(month):
```

1. Adapted data types: Decimal stores fat, SNF, rates and money accurately to decimal places.
2. Seasonal rate: April, May and June use a base rate of ₹32.00 and fat step value of ₹0.45. Other months use the standard rates.
3. Precision: Fat and SNF readings accept two decimal places, such as 4.25% and 8.65%.
4. Bonus rule: A bonus of ₹0.50 per litre is applied to all ten-day litres only when the total is strictly greater than 500 litres.
5. Validation: The program rejects negative litres, invalid fat/SNF ranges, and invalid month numbers.
6. Testing: Test April and July, exactly 500 litres, more than 500 litres, and fat/SNF values with two decimal places.
7. Complexity: Each payment calculation takes O(1) time and O(1)extra space.
8. Design recommendation: Keep the rate values as named constants and use Decimal for financial calculations. This makes future rate changes easier and helps avoid floating-point errors.

Assumption: The input is litres supplied per day, so ten-day litres are calculated by multiplying by 10. If the entered quantity is already the ten-day total, remove that multiplication.